complete java class code example of each combination of intermediate and terminal operations

filter() boolean checks + foreach() logging

filter() null checks + foreach() logging

filter() comparisons + foreach() logging

filter() method calls returning boolean + foreach() logging

distinct() rely on equals() / hashcode() + foreach() logging

distinct() immutable objects + foreach() logging

map() transform objects + foreach() logging

map() create new objects + foreach() logging

map() call pure functions + foreach() logging

flatmap() return stream + foreach() logging

flatmap() split data + foreach() logging

flatmap() expand collections + foreach() logging

sorted() natural ordering + foreach() logging

sorted() custom comparator + foreach() logging

sorted() multi-field compare + foreach() logging

peek() logging + foreach() logging

peek() debugging + foreach() logging

peek() metrics + foreach() logging

Below are **all 18 programs converted to Java 1.5 compatible code**.

Since Java 1.5:

- ❌ No Stream API
- ❌ No Optional
- ❌ No lambda expressions
- ❌ No method references
- ❌ No List.of(), Map.of()
- ❌ No Objects.hash()
- ❌ No Comparator.comparing()

We replace everything with:

- ✔ for-each loops
- ✔ Iterator
- ✔ Collections / Arrays
- ✔ HashSet for distinct
- ✔ Manual sorting with Comparator
- ✔ Null checks
- ✔ Primitive counters instead of AtomicInteger

Below are **complete, standalone Java class examples** where **each pipeline ends with forEach() used purely for logging**.

Key rules followed:

- **One intermediate concept per class**
- **forEach() is the terminal operation**
- Logging is explicit (System.out.println)
- Java **8+**, interview-safe
- No collectors, no shortcuts

---

1 filter() boolean checks + forEach() Logging

```
import java.util.*;

public class FilterBooleanForEachLogging {
   public static void main(String[] args) {
      List<Integer> numbers = List.of(10, 15, 20, 25);

      numbers.stream()
            .filter(n -> n % 2 == 0)                    // boolean check
            .forEach(n -> System.out.println("Even: " + n));
   }
}
```

---

2 filter() null checks + forEach() Logging

```
import java.util.*;

public class FilterNullForEachLogging {
   public static void main(String[] args) {
      List<String> names = Arrays.asList("Anurag", null, "Amit");

      names.stream()
            .filter(Objects::nonNull)
            .forEach(name -> System.out.println("Name: " + name));
   }
}
```

---

3 filter() comparisons + forEach() Logging

```
import java.util.*;

public class FilterComparisonForEachLogging {
   public static void main(String[] args) {
      List<Integer> salaries = List.of(20000, 40000, 60000);

      salaries.stream()
            .filter(s -> s >= 30000)
            .forEach(s -> System.out.println("Eligible salary: " + s));
   }
}
```

---

4 filter() method call boolean + forEach() Logging

```
import java.util.*;

public class FilterMethodForEachLogging {

   static class User {
      String name;
      boolean active;

      User(String name, boolean active) {
         this.name = name;
         this.active = active;
      }

      boolean isActive() {
         return active;
      }
   }

   public static void main(String[] args) {
      List<User> users = List.of(
            new User("Anurag", true),
            new User("Ravi", false)
      );

      users.stream()
            .filter(User::isActive)
            .forEach(u -> System.out.println("Active user: " + u.name));
   }
}
```

---

5 distinct() equals/hashCode + forEach() Logging

```
import java.util.*;

public class DistinctEqualsForEachLogging {

    static class Person {
        String name;

        Person(String name) {
            this.name = name;
        }

        @Override
        public boolean equals(Object o) {
            return o instanceof Person p && name.equals(p.name);
        }

        @Override
        public int hashCode() {
            return Objects.hash(name);
        }
    }

    public static void main(String[] args) {
        List<Person> people = List.of(
                new Person("Anurag"),
                new Person("Anurag"),
                new Person("Ravi")
        );

        people.stream()
                .distinct()
                .forEach(p -> System.out.println("Unique person: " + p.name));
    }
}
```

---

6 distinct() immutable objects + forEach() Logging

```
import java.util.*;

public class DistinctImmutableForEachLogging {
   public static void main(String[] args) {
      List<String> languages = List.of("Java", "Java", "Python");

      languages.stream()
            .distinct()
            .forEach(lang -> System.out.println("Language: " + lang));
   }
}
```

---

7 map() transform objects + forEach() Logging

```
import java.util.*;

public class MapTransformForEachLogging {
   public static void main(String[] args) {
      List<String> names = List.of("anurag", "amit");

      names.stream()
            .map(String::toUpperCase)
            .forEach(name -> System.out.println("Upper: " + name));
   }
}
```

---

8 map() create new objects + forEach() Logging

```
import java.util.*;

public class MapNewObjectForEachLogging {

   static class UserDTO {
      String name;

      UserDTO(String name) {
         this.name = name;
      }
   }

   public static void main(String[] args) {
      List<String> names = List.of("Anurag", "Amit");

      names.stream()
            .map(UserDTO::new)
            .forEach(dto -> System.out.println("DTO created: " + dto.name));
   }
}
```

---

9 map() pure functions + forEach() Logging

```
import java.util.*;

public class MapPureFunctionForEachLogging {

   static int square(int n) {
      return n * n;
   }

   public static void main(String[] args) {
      List<Integer> numbers = List.of(2, 3, 4);

      numbers.stream()
            .map(MapPureFunctionForEachLogging::square)
            .forEach(n -> System.out.println("Square: " + n));
   }
}
```

---

10 flatMap() return Stream + forEach() Logging

```
import java.util.*;

public class FlatMapStreamForEachLogging {
   public static void main(String[] args) {
      List<List<Integer>> lists = List.of(
            List.of(1, 2),
            List.of(3, 4)
      );

      lists.stream()
            .flatMap(List::stream)
            .forEach(n -> System.out.println("Value: " + n));
   }
}
```

---

11 flatMap() split data + forEach() Logging

```
import java.util.*;

public class FlatMapSplitForEachLogging {
   public static void main(String[] args) {
      List<String> sentences = List.of("Java Stream", "Flat Map");

      sentences.stream()
            .flatMap(s -> Arrays.stream(s.split(" ")))
            .forEach(word -> System.out.println("Word: " + word));
   }
}
```

---

12 flatMap() expand collections + forEach() Logging

```
import java.util.*;

public class FlatMapExpandForEachLogging {
   public static void main(String[] args) {
      Map<String, List<String>> data = Map.of(
            "A", List.of("x", "y"),
            "B", List.of("z")
      );

      data.values().stream()
            .flatMap(Collection::stream)
            .forEach(v -> System.out.println("Value: " + v));
```

```
    }
}
```

---

13 sorted() natural ordering + forEach() Logging

```
import java.util.*;

public class SortedNaturalForEachLogging {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(5, 1, 3);

        numbers.stream()
                .sorted()
                .forEach(n -> System.out.println("Sorted: " + n));
    }
}
```

---

14 sorted() custom comparator + forEach() Logging

```
import java.util.*;

public class SortedCustomForEachLogging {
    public static void main(String[] args) {
        List<String> names = List.of("Anurag", "Ravi", "Amit");

        names.stream()
                .sorted(Comparator.reverseOrder())
                .forEach(name -> System.out.println("Name: " + name));
    }
}
```

---

15 sorted() multi-field + forEach() Logging

```
import java.util.*;

public class SortedMultiFieldForEachLogging {

   static class Employee {
      String name;
      int age;

      Employee(String name, int age) {
         this.name = name;
         this.age = age;
      }
   }

   public static void main(String[] args) {
      List<Employee> employees = List.of(
            new Employee("Anurag", 30),
            new Employee("Anurag", 25),
            new Employee("Ravi", 35)
      );

      employees.stream()
            .sorted(Comparator
                  .comparing((Employee e) -> e.name)
                  .thenComparingInt(e -> e.age))
            .forEach(e ->
                  System.out.println(e.name + " - " + e.age));
   }
}
```

---

16 peek() logging + forEach() Logging

```
import java.util.*;

public class PeekLoggingForEachLogging {
   public static void main(String[] args) {
      List.of(1, 2, 3).stream()
            .peek(n -> System.out.println("Peek log: " + n))
            .forEach(n -> System.out.println("Final: " + n));
   }
}
```

---

17 peek() debugging + forEach() Logging

```
import java.util.*;

public class PeekDebugForEachLogging {
   public static void main(String[] args) {
      List.of(10, 20, 30).stream()
            .peek(n -> {
               if (n == 20) {
                  System.out.println("Debug breakpoint hit: " + n);
               }
            })
            .forEach(n -> System.out.println("Processed: " + n));
   }
}
```

---

18 peek() metrics + forEach() Logging

```
import java.util.*;
import java.util.concurrent.atomic.AtomicInteger;

public class PeekMetricsForEachLogging {
   public static void main(String[] args) {
      AtomicInteger counter = new AtomicInteger();

      List.of(5, 10, 15).stream()
            .peek(n -> counter.incrementAndGet())
            .forEach(n -> System.out.println("Value: " + n));

      System.out.println("Total processed: " + counter.get());
   }
}
```